

Introduction to Python Programming

APRIL 4

MIT, Kundapura



Module-1

PYTHON BASICS

Python Basics: Entering Expressions into the Interactive Shell, The Integer, Floating-Point, and String Data Types, String Concatenation and Replication, Storing Values in Variables, Your First Program, Dissecting Your Program,

Flow control: Boolean Values, Comparison Operators, Boolean Operators, Mixing Boolean and Comparison Operators, Elements of Flow Control, Program Execution, Flow Control Statements, Importing Modules, Ending a Program Early with `sys.exit()`,

Functions: `def` Statements with Parameters, Return Values and `return` Statements, The `None` Value, Keyword Arguments and `print()`, Local and Global Scope, The `global` Statement, Exception Handling, A Short Program: Guess the Number

Textbook 1: Chapters 1 – 3

Module-1

1.1 PYTHON BASICS:

1.1.1 Entering Expressions into the Interactive Shell:

- You run the interactive shell by launching IDLE.
 - A window with the >>> prompt should appear; that's the interactive shell.
 - Enter 2 + 2 at the prompt to have Python do some simple math.
- >>> 2 + 2
- 4
- In Python, 2 + 2 is called an expression, which is the most basic kind of programming instruction in the language.
 - Expressions consist of values (such as 2) and operators (such as +), and they can always evaluate (that is, reduce) down to a single value.
 - That means you can use expressions anywhere in Python code that you could also use a value.
 - In the previous example, 2 + 2 is evaluated down to a single value, 4.
 - A single value with no operators is also considered an expression, though it evaluates only to itself, as shown here

>>> 2

2

There are plenty of other operators you can use in Python expressions, too. For example, Table 1-1 lists all the math operators in Python.

Table 1-1: Math Operators from Highest to Lowest Precedence

Table 1-1: Math Operators from Highest to Lowest Precedence

Operator	Operation	Example	Evaluates to...
**	Exponent	2 ** 3	8
%	Modulus/remainder	22 % 8	6
//	Integer division/floored quotient	22 // 8	2
/	Division	22 / 8	2.75
*	Multiplication	3 * 5	15
-	Subtraction	5 - 2	3
+	Addition	2 + 2	4

order of operations (also called precedence) of Python math operators is similar to that of mathematics.

The `**` operator is evaluated first;

the `%`, `//`, `/`, `*`, operators are evaluated next, from left to right;

and the `-` and `+` operators are evaluated last (also from left to right).

You can use parentheses to override the usual precedence if you need to. Enter the following expressions into the interactive shell:

```
>>> 2 + 3 * 6
```

```
20
```

```
>>> (2 + 3) * 6
```

```
30
```

```
>>> 48565878 * 578453
```

```
28093077826734
```

```
>>> 2 ** 8
```

```
256
```

```
>>> 23 / 7
```

```
3.2857142857142856
```

```
>>> 23 // 7
```

```
3
```

```
>>> 23 % 7
```

```
2
```

```
>>> 2 + 2
```

```
4
```

```
>>> (5 - 1) * ((7 + 1) / (3 - 1))
```

```
16.0
```

In each case, you as the programmer must enter the expression, but Python does the hard part of evaluating it down to a single value. Python will keep evaluating parts of the expression until it becomes a single value.

1.1.2 The Integer, Floating-Point, and String Data Types

- We know that expressions are just values combined with operators, and they always evaluate down to a single value.
- A data type is a category for values, and every value belongs to exactly one data type.
- The values -2 and 30, for example, are said to be integer values.
- The integer (or int) data type indicates values that are whole numbers.
- Numbers with a decimal point, such as 3.14, are called floating-point numbers (or floats).
- Note that even though the value 42 is an integer, the value 42.0 would be a floating-point number.
- Python programs can also have text values called strings, or strs
- Always surround your string in single quote (') characters (as in 'Hello' or 'Goodbye cruel world!') so Python knows where the string begins and ends.
- You can even have a string with no characters in it, "", called a blank string.

Table 1-2: Common Data Types

Data type	Examples
Integers	-2, -1, 0, 1, 2, 3, 4, 5
Floating-point numbers	-1.25, -1.0, --0.5, 0.0, 0.5, 1.0, 1.25
Strings	'a', 'aa', 'aaa', 'Hello!', '11 cats'

1.1.3 String Concatenation and Replication

String Concatenation

- The meaning of an operator may change based on the data types of the values next to it.
- For example, + is the addition operator when it operates on two integers or floating-point values.
- However, when + is used on two string values, it joins the strings as the string concatenation operator.

- For Example

```
>>> 'Hello' + 'SMVITM'
```

```
'HelloSMVITM'
```

-
- The expression evaluates down to a single, new string value that combines the text of the two strings.
 - However, if you try to use the + operator on a string and an integer value, Python will not know how to handle this, and it will display an error message.

```
>>> 'Hello' + 42
```

```
TypeError: Can't convert 'int' object to str implicitly
```

String Replication

- The * operator is used for multiplication when it operates on two integer or floating-point values.
- But when the * operator is used on one string value and one integer value, it becomes the string replication operator.
- For Example, enter a string multiplied by a number

```
>>> 'Hello' * 5
```

```
'HelloHelloHelloHelloHello'
```

- The expression evaluates down to a single string value that repeats the original a number of times equal to the integer value.
- String replication is a useful trick, but it's not used as often as string concatenation.
- The * operator can be used with only two numeric values (for multiplication) or one string value and one integer value (for string replication).
- Otherwise, Python will just display an error message.

```
>>> 'Hello' * 'SMVITM'
```

```
'Hello' * 'SMVITM' TypeError: can't multiply sequence by non-int of type 'str'
```

```
>>> 'Hello' * 5.0
```

```
'Hello' * 5.0 TypeError: can't multiply sequence by non-int of type 'float'
```

1.1.4 Storing Values in Variables

- A variable is like a box in the computer's memory where you can store a single value.
- If you want to use the result of an evaluated expression later in your program, you can save it inside a variable.

Assignment Statements

- You'll store values in variables with an assignment statement.
- An assignment statement consists of a variable name, an equal sign (called the assignment operator), and the value to be stored.
- If you enter the assignment statement `spam = 42`, then a variable named `spam` will have the integer value 42 stored in it.

Think of a variable as a labeled box that a value is placed in, as in Figure 1-2.

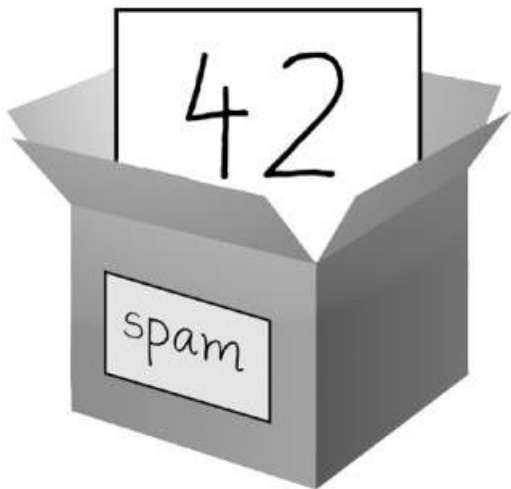


Figure 1-2: `spam = 42` is like telling the program, "The variable `spam` now has the integer value 42 in it."

For example, enter the following into the interactive shell:

```
❶ >>> spam = 40
```

```
>>> spam
```

```
40
```

```
>>> eggs = 2
```

❷ >>> spam + eggs

42

>>> spam + eggs + spam

82

❸ >>> spam = spam + 2

>>> spam

42

- A variable is initialized (or created) the first time a value is stored in it ❶.
- After that, you can use it in expressions with other variables and values ❷.
- When a variable is assigned a new value ❸, the old value is forgotten, which is why spam evaluated to 42 instead of 40 at the end of the example.
- This is called overwriting the variable.

Enter the following code into the interactive shell to try overwriting a string:

```
>>> spam = 'Hello'
```

```
>>> spam
```

```
'Hello'
```

```
>>> spam = 'Goodbye'
```

```
>>> spam
```

```
'Goodbye'
```

Just like the box in Figure 1-3, the spam variable in this example stores 'Hello' until you replace it with 'Goodbye'.

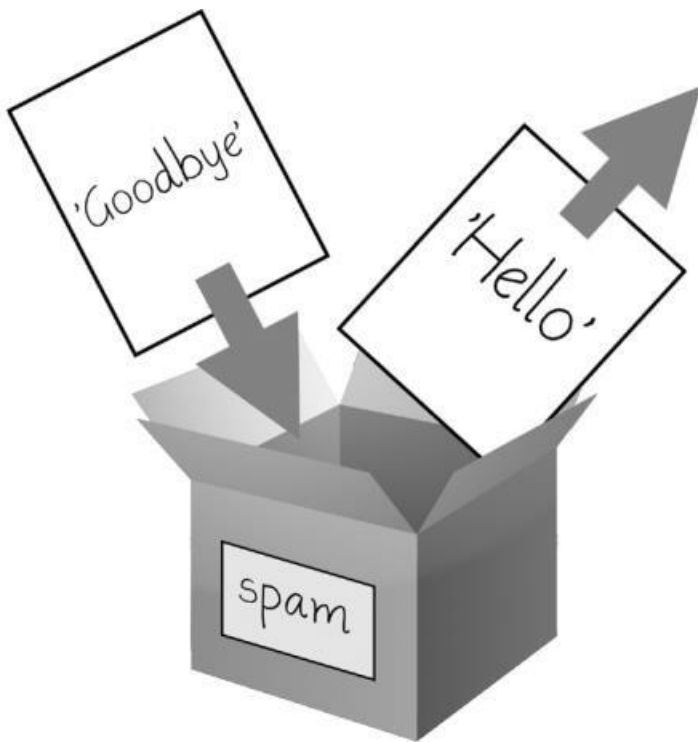


Figure 1-3: When a new value is assigned to a variable, the old one is forgotten.

Variable Names

Table 1-3 has examples of legal variable names.

You can name a variable anything as long as it obeys the following three rules:

- 1. It can be only one word.**
- 2. It can use only letters, numbers, and the underscore (_) character.**
- 3. It can't begin with a number.**

Table 1-3: Valid and Invalid Variable Name

Valid variable names	Invalid variable names
balance	current-balance (hyphens are not allowed)
currentBalance	current balance (spaces are not allowed)
current_balance	4account (can't begin with a number)
_spam	42 (can't begin with a number)
SPAM	total_\$um (special characters like \$ are not allowed)
account4	'hello' (special characters like ' are not allowed)

- Variable names are case-sensitive, meaning that spam, SPAM, Spam, and sPaM are four different variables.
- It is a Python convention to start your variables with a lowercase letter.
- A good variable name describes the data it contains.

Variable names with more than one word can be difficult to read.

There are several techniques you can use to make them more readable:

- **Camel Case:** Each word, except the first, starts with a capital letter
`myVariableName = "Udupi"`
- **Pascal Case:** Each word starts with a capital letter
`MyVariableName = "Mangalore"`
- **Snake Case:** Each word is separated by an underscore character
`my_variable_name = "Bangalore"`

1.1.5 Your First Program

- ❶ # This program says hello and asks for my name.
- ❷ `print('Hello world!')`
`print('What is your name?') # ask for their name`
- ❸ `myName = input() # Hithesh`
- ❹ `print('It is good to meet you, ' + myName) # concate`
- ❺ `print('The length of your name is:')`
`print(len(myName))`
- ❻ `print('What is your age?') # ask for their age`
`myAge = input()`
`print('You will be ' + str(int(myAge) + 1) + ' in a year.')`

Output:

```
Python 3.3.2 (v3.3.2:d047928ae3f6, May 16 2013, 00:06:53) [MSC v.1600 64 bit
(AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
Hello world!
What is your name?
Al
It is good to meet you, Al
The length of your name is:
2
What is your age?
4
You will be 5 in a year.
>>>
```

When there are no more lines of code to execute, the Python program terminates; that is, it stops running. (You can also say that the Python program exits.)

1.1.6 Dissecting Your Program

Comments

The following line is called a comment.

❶ **# This program says hello and asks for my name.**

- Python ignores comments, and you can use them to write notes or remind yourself what the code is trying to do.
- Any text for the rest of the line following a hash mark (#) is part of a comment.
- Sometimes, programmers will put a # in front of a line of code to temporarily remove it while testing a program.
- This is called commenting out code, and it can be useful when you're trying to figure out why a program doesn't work.
- You can remove the # later when you are ready to put the line back in.
- Python also ignores the blank line after the comment.
- You can add as many blank lines to your program as you want. This can make your code easier to read, like paragraphs in a book.

The print() Function

The print() function displays the string value inside the parentheses on the screen.

❷ **print('Hello world!')**

print('What is your name?') # ask for their name

- The line `print('Hello world!')` means “Print out the text in the string 'Hello world!'.”
- When Python executes this line, you say that Python is calling the `print()` function and the string value is being passed to the function.
- A value that is passed to a function call is an argument. Notice that the quotes are not printed to the screen.
- They just mark where the string begins and ends; they are not part of the string value.

The input() Function

The input() function waits for the user to type some text on the keyboard and press enter.

❸ **myName = input()**

- This function call evaluates to a string equal to the user's text, and the previous line of code assigns the myName variable to this string value.
- You can think of the input() function call as an expression that evaluates to whatever string the user typed in.
- If the user entered 'Al', then the expression would evaluate to myName = 'Al'

Printing the User's Name

The following call to print() actually contains the expression 'It is good to meet you, ' + myName between the parentheses.

❹ **print('It is good to meet you, ' + myName)**

- Remember that expressions can always evaluate to a single value.
- If 'Al' is the value stored in myName on the previous line, then this expression evaluates to 'It is good to meet you, Al'.
- This single string value is then passed to print(), which prints it on the screen.

The len() Function

You can pass the len() function a string value (or a variable containing a string), and the function evaluates to the integer value of the number of characters in that string.

❺ **print('The length of your name is:')**
print(len(myName))

Enter the following into the interactive shell to try this

```
>>> len('hello')
```

```
5
```

```
>>> len('print() allows you to pass it either integer values or string values. ')
```

```
68
```

```
>>> len("")
```

```
0
```

-
- Just like those examples, `len(myName)` evaluates to an integer.
 - It is then passed to `print()` to be displayed on the screen.
 - Notice that `print()` allows you to pass it either integer values or string values.
- But notice the error that shows up when you type the following into the interactive shell:
- ```
>>> print('I am ' + 29 + ' years old.')
```

**TypeError: Can't convert 'int' object to str implicitly**

- Python gives an error because you can use the `+` operator only to add two integers together or concatenate two strings.
- You can't add an integer to a string because this is ungrammatical in Python.

### **The `str()`, `int()`, and `float()` Functions**

- If you want to concatenate an integer such as 29 with a string to pass to `print()`, you'll need to get the value '29', which is the string form of 29.
- The `str()` function can be passed an integer value and will evaluate to a string value version of it, as follows

```
>>> str(29)
'29'
>>> print('I am ' + str(29) + ' years old.')
```

**I am 29 years old.**

- Because `str(29)` evaluates to '29', the expression `'I am ' + str(29) + ' years old.'` evaluates to `'I am ' + '29' + ' years old.'`, which in turn evaluates to `'I am 29 years old.'`.
- This is the value that is passed to the `print()` function.

The `str()`, `int()`, and `float()` functions will evaluate to the string, integer, and floating-point forms of the value you pass, respectively.

Try converting some values in the interactive shell with these functions:

```
>>> str(0)
'0'
```

---

```
>>> str(-3.14)
```

```
'-3.14'
```

```
>>> int('42')
```

```
42
```

```
>>> int('-99')
```

```
-99
```

```
>>> int(1.25)
```

```
1
```

```
>>> int(1.99)
```

```
1
```

```
>>> float('3.14')
```

```
3.14
```

```
>>> float(10)
```

```
10.0
```

- The `str()` function is handy when you have an integer or float that you want to concatenate to a string.
- The `int()` function is also helpful if you have a number as a string value that you want to use in some mathematics.
- For example, the `input()` function always returns a string, even if the user enters a number.
- Enter `spam = input()` into the interactive shell and enter 101 when it waits for your
- text

```
>>> spam = input()
```

```
101
```

```
>>> spam
```

```
'101'
```

The value stored inside `spam` isn't the integer 101 but the string '101'.

If you want to do math using the value in `spam`, use the `int()` function to get the integer form of `spam` and then store this as the new value in `spam`.

---

```
>>> spam = int(spam)
```

```
>>> spam
```

```
101
```

Now spam variable as an integer instead of a string.

In your program, you used the `int()` and `str()` functions in the last three lines to get a value of the appropriate data type for the code.

## ⑥ `print('What is your age?') # ask for their age`

```
myAge = input()
```

```
print('You will be ' + str(int(myAge) + 1) + ' in a year.')
```

- The `myAge` variable contains the value returned from `input()`.
- Because the `input()` function always returns a string (even if the user typed in a number), you can use the `int(myAge)` code to return an integer value of the string in `myAge`.
- This integer value is then added to 1 in the expression `int(myAge) + 1`.
- The result of this addition is passed to the `str()` function: `str(int(myAge) + 1)`.
- The string value returned is then concatenated with the strings `'You will be '` and `' in a year.'` to evaluate to one large string value.
- This large string is finally passed to `print()` to be displayed on the screen.

## Practice Questions

1. Which of the following are operators, and which are values?

\*

'hello'

-88.8

-

/

+

5

2. Which of the following is a variable, and which is a string?

spam

'spam'

- 
3. Name three data types.
  4. What is an expression made up of? What do all expressions do?
  5. This chapter introduced assignment statements, like `spam = 10`. What is the difference between an expression and a statement?
  6. What does the variable `bacon` contain after the following code runs?  
`bacon = 20`  
`bacon + 1`
  7. What should the following two expressions evaluate to?  
`'spam' + 'spamspam'`  
`'spam' * 3`
  8. Why is `eggs` a valid variable name while `100` is invalid?
  9. What three functions can be used to get the integer, floating-point number, or string version of a value?
  10. Why does this expression cause an error? How can you fix it?  
`'I have eaten ' + 99 + ' burritos.'`

## Module-1

### 1.2 FLOW CONTROL:

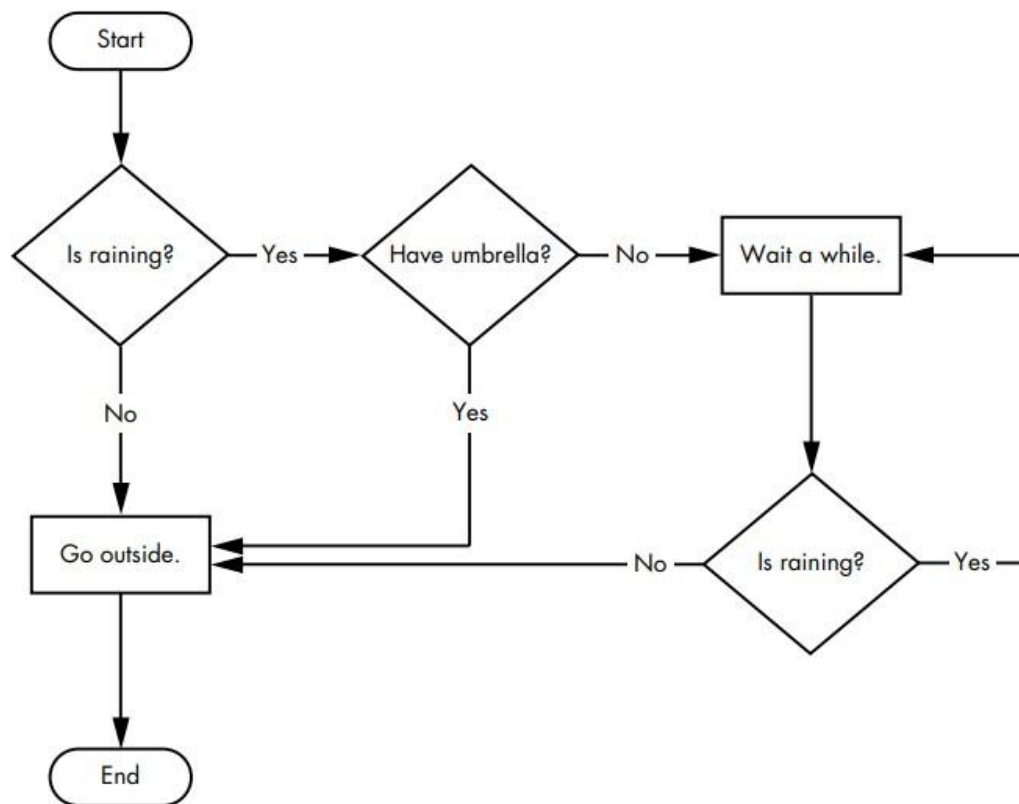


Figure 2-1: A flowchart to tell you what to do if it is raining

- In a flowchart, there is usually more than one way to go from the start to the end.
- The same is true for lines of code in a computer program.
- Flowcharts represent these branching points with diamonds, while the other steps are represented with rectangles.
- The starting and ending steps are represented with rounded rectangles.

#### 1.2.1 Boolean Values

- While the integer, floating-point, and string data types have an unlimited number of possible values, the Boolean data type has only two values: True and False.

- When typed as Python code, the Boolean values True and False lack the quotes you place around strings, and they always start with a capital T or F, with the rest of the word in lowercase.

- Enter the following into the interactive shell.

```
❶ >>> spam = True
>>> spam
True
❷ >>> true
Traceback (most recent call last):
 File "<pyshell#2>", line 1, in <module>
 true
NameError: name 'true' is not defined
❸ >>> True = 2 + 2
SyntaxError: assignment to keyword
```

Like any other value, Boolean values are used in expressions and can be stored in variables ❶. If you don't use the proper case ❷ or you try to use True and False for variable names ❸, Python will give you an error message.

## 1.2.2 Comparison Operators

- Comparison operators compare two values and evaluate down to a single Boolean value.
- Table 2-1 lists the comparison operators

**Table 2-1:** Comparison Operators

| Operator | Meaning                  |
|----------|--------------------------|
| ==       | Equal to                 |
| !=       | Not equal to             |
| <        | Less than                |
| >        | Greater than             |
| <=       | Less than or equal to    |
| >=       | Greater than or equal to |

These operators evaluate to True or False depending on the values you give them. Let's try some operators now, starting with == and !=.

```
>>> 42 == 42
```

```
True
```

```
>>> 42 == 99
```

---

False

```
>>> 2 != 3
```

True

```
>>> 2 != 2
```

False

- == (equal to) evaluates to True when the values on both sides are the same.
- != (not equal to) evaluates to True when the two values are different.
- The == and != operators can actually work with values of any data type.

---

```
>>> 'hello' == 'hello'
True
>>> 'hello' == 'Hello'
False
>>> 'dog' != 'cat'
True
>>> True == True
True
>>> True != False
True
>>> 42 == 42.0
True
❶ >>> 42 == '42'
False
```

---

Note that an integer or floating-point value will always be unequal to a string value. The expression `42 == '42'` ❶ evaluates to False because Python considers the integer 42 to be different from the string '42'.

The <, >, <=, and >= operators, on the other hand, work properly only with integer and floating-point values.

---

```
>>> 42 < 100
True
>>> 42 > 100
False
>>> 42 < 42
False
>>> eggCount = 42
❶ >>> eggCount <= 42
True
>>> myAge = 29
❷ >>> myAge >= 10
True
```

---

You'll often use comparison operators to compare a variable's value to some other value, like in the `eggCount <= 42` ❶ and `myAge >= 10` ❷ examples. (After all, instead of typing `'dog' != 'cat'` in your code, you could have just typed `True`.) You'll see more examples of this later when you learn about flow control statements.

## 1.2.3 Boolean Operators

- The three Boolean operators **and**, **or**, and **not** are used to compare Boolean values.
- Like comparison operators, they evaluate these expressions down to a Boolean value.
- Let's explore these operators in detail, starting with the **and** operator

### Binary Boolean Operators

- The **and** and **or** operators always take two Boolean values (or expressions), so they're considered binary operators.

#### **and** operator:

- The **and** operator evaluates an expression to `True` if both Boolean values are `True`; otherwise, it evaluates to `False`.
- Enter some expressions using **and** into the interactive shell.

---

```
>>> True and True
True
>>> True and False
False
```

---

A *truth table* shows every possible result of a Boolean operator. Table 2-2 is the truth table for the **and** operator.

**Table 2-2:** The **and** Operator's Truth Table

| Expression      | Evaluates to... |
|-----------------|-----------------|
| True and True   | True            |
| True and False  | False           |
| False and True  | False           |
| False and False | False           |

### or operator

- The **or** operator evaluates an expression to True if either of the two Boolean values is True. If both are False, it evaluates to False.

---

```
>>> False or True
True
>>> False or False
False
```

---

- You can see every possible outcome of the **or** operator in its truth table, shown in Table 2-3

**Table 2-3:** The **or** Operator's Truth Table

| Expression     | Evaluates to... |
|----------------|-----------------|
| True or True   | True            |
| True or False  | True            |
| False or True  | True            |
| False or False | False           |

### The not Operator

Unlike **and** and **or**, the **not** operator operates on only one Boolean value (or expression).

The **not** operator simply evaluates to the opposite Boolean value.

---

```
>>> not True
False
❶ >>> not not not not True
True
```

---

Much like using double negatives in speech and writing, you can nest not operators ❶, though there's never not no reason to do this in real programs. Table 2-4 shows the truth table for not.

**Table 2-4:** The not Operator's Truth Table

| Expression | Evaluates to... |
|------------|-----------------|
| not True   | False           |
| not False  | True            |

## 1.2.4 Mixing Boolean and Comparison Operators

- Since the comparison operators evaluate to Boolean values, you can use them in expressions with the Boolean operators.
- The **and**, **or** and **not** operators are called Boolean operators because they always operate on the Boolean values True and False.
- While expressions like `4 < 5` aren't Boolean values, they are expressions that evaluate down to Boolean values.
- Try entering some Boolean expressions that use comparison operators into the interactive shell.

---

```
>>> (4 < 5) and (5 < 6)
True
>>> (4 < 5) and (9 < 6)
False
>>> (1 == 2) or (2 == 2)
True
```

---

The computer will evaluate the left expression first, and then it will evaluate the right expression. When it knows the Boolean value for each, it will then evaluate the whole expression down to one Boolean value. You can think of the computer's evaluation process for `(4 < 5) and (5 < 6)` as shown in Figure 2-2.

You can also use multiple Boolean operators in an expression, along with the comparison operators.

---

```
>>> 2 + 2 == 4 and not 2 + 2 == 5 and 2 * 2 == 2 + 2
True
```

---

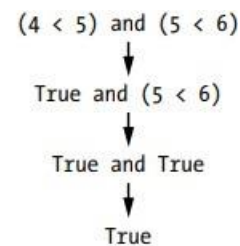


Figure 2-2: The process of evaluating `(4 < 5) and (5 < 6)` to `True`.

The Boolean operators have an order of operations just like the math operators do. After any math and comparison operators evaluate, Python evaluates the `not` operators first, then the `and` operators, and then the `or` operators.

## 1.2.5 Elements of Flow Control

Flow control statements often start with a part called the condition, and all are followed by a block of code called the clause.

### Conditions

- The Boolean expressions are also considered as conditions, which are the same thing as expressions; condition is just a more specific name in the context of flow control statements.
- Conditions always evaluate down to a Boolean value, `True` or `False`.

- 
- A flow control statement decides what to do based on whether its condition is True or False, and almost every flow control statement uses a condition.

### Blocks of Code

- Lines of Python code can be grouped together in blocks.
- You can tell when a block begins and ends from the indentation of the lines of code.
- There are three rules for blocks.

1. Blocks begin when the indentation increases.
2. Blocks can contain other blocks.
3. Blocks end when the indentation decreases to zero or to a containing block's indentation.

Blocks are easier to understand by looking at some indented code, so let's find the blocks in part of a small game program, shown here:

---

```
if name == 'Mary':
 ❶ print('Hello Mary')
 if password == 'swordfish':
 ❷ print('Access granted.')
 else:
 ❸ print('Wrong password.')
```

---

The first block of code ❶ starts at the line `print('Hello Mary')` and contains all the lines after it. Inside this block is another block ❷, which has only a single line in it: `print('Access Granted.')`. The third block ❸ is also one line long: `print('Wrong password.')`.

## 1.2.6 Program Execution

- Simple python program usually executes instructions at the top of the program going down, one after another.
- The program execution (or simply, execution) is a term for the current instruction being executed.
- If you print the source code on paper and put your finger on each line as it is executed, you can think of your finger as the program execution.
- Not all programs execute by simply going straight down, however.

- 
- If you use your finger to trace through a program with flow control statements, you'll likely find yourself jumping around the source code based on conditions, and you'll probably skip entire clauses.

## 1.2.7 Flow Control Statements

### if Statements

- The most common type of flow control statement is the if statement.
- An if statement's clause (that is, the block following the if statement) will execute if the statement's condition is True.
- The clause is skipped if the condition is False.
- In plain English, an if statement could be read as, "If this condition is true, execute the code in the clause."

Syntax:

**if condition:**

**# body of if statement (if clause)**

In Python, an if statement consists of the following:

- The **if** keyword
- A condition (that is, an expression that evaluates to True or False)
- A colon
- Starting on the next line, an **indented block of code** (called the if clause)

**Example:**

```
name = 'alice'
```

```
if name == 'Alice' :
```

```
 print('Hi, Alice. ') # clause(block of code)
```

```
print("not Alice")
```

All flow control statements end with a colon and are followed by a new block of code (the clause). This if statement's clause is the block with `print('Hi, Alice.')`. Figure 2-3 shows what a flowchart of this code would look like.

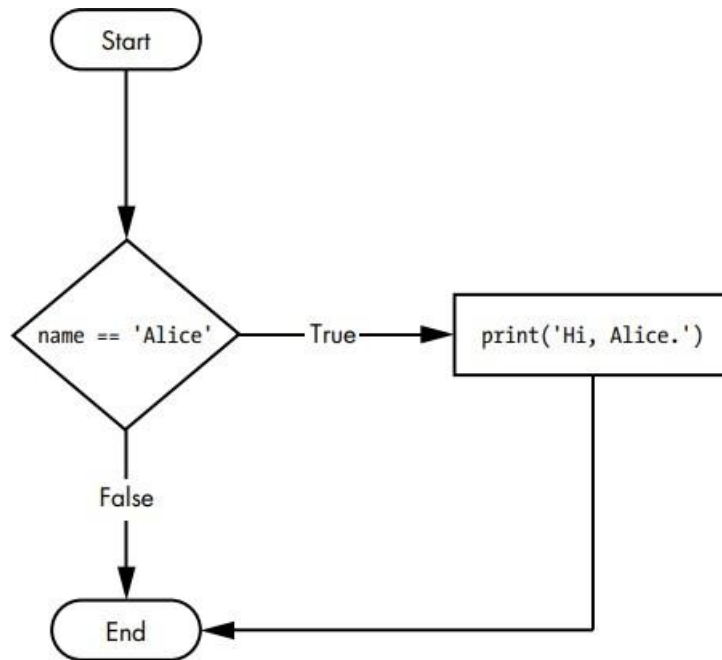


Figure 2-3: The flowchart for an if statement

### else Statements

- An if clause can optionally be followed by an else statement.
- The else clause is executed only when the if statement's condition is False.
- In plain English, an else statement could be read as, "If this condition is true, execute this code. Or else, execute that code."

#### Syntax:

**if condition:**

**# body of if statement**

**else:**

**# body of else statement**

- An else statement doesn't have a condition, and in code, an else statement always consists of the following:

- 
- The else keyword
  - A colon
  - Starting on the next line, an indented block of code (called the else clause)

Returning to the Alice example, let's look at some code that uses an else statement to offer a different greeting if the person's name isn't Alice.

```
name = 'Alice'
if name == 'alice' :
 print('Hi, Alice. ') # if clause(block of code)
else:
 print('Hello stranger') # else clause(block of code)
```

Figure 2-4 shows what a flowchart of this code would look like.

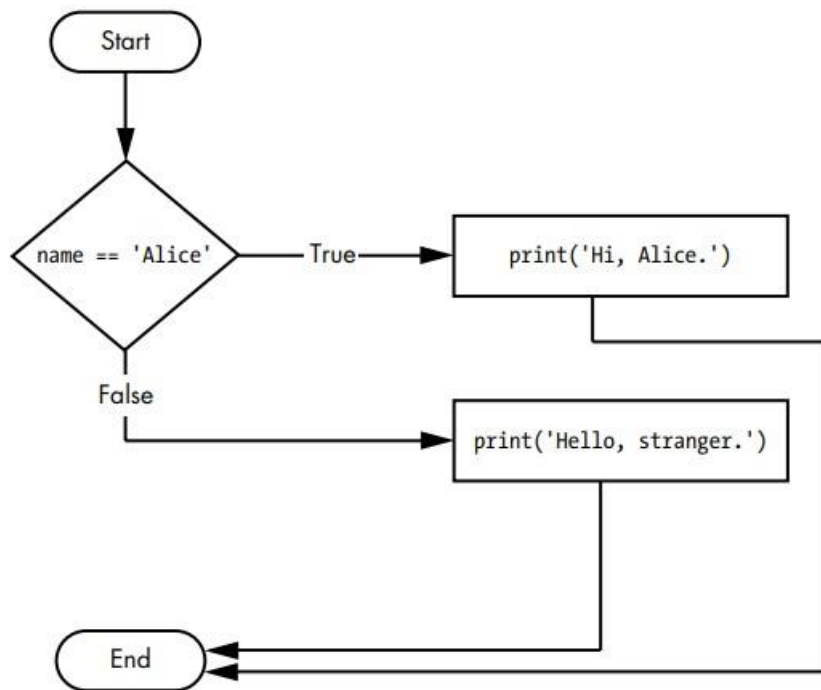


Figure 2-4: The flowchart for an *else* statement

## elif Statements

- While only one of the if or else clauses will execute, you may have a case where you want one of many possible clauses to execute.
- The elif statement is an “else if” statement that always follows an if or another elif statement.
- It provides another condition that is checked only if any of the previous conditions were False.

### Syntax:

**if condition1:**

**# code block 1**

**elif condition2:**

**# code block 2**

---

**else:**

**# code block**

In code, an elif statement always consists of the following:

- The elif keyword
- A condition (that is, an expression that evaluates to True or False)
- A colon
- Starting on the next line, an indented block of code (called the elif clause)

Let's add an elif to the name checker to see this statement in action.

```
name = 'alice'
age = 10
if name == 'Alice' :
 print('Hi, Alice. ') # clause(block of code)
elif age<12:
 print('You are not Alice, Kiddo')
```

This time, you check the person's age, and the program will tell them something different if they're younger than 12. You can see the flowchart for this in Figure 2-5.

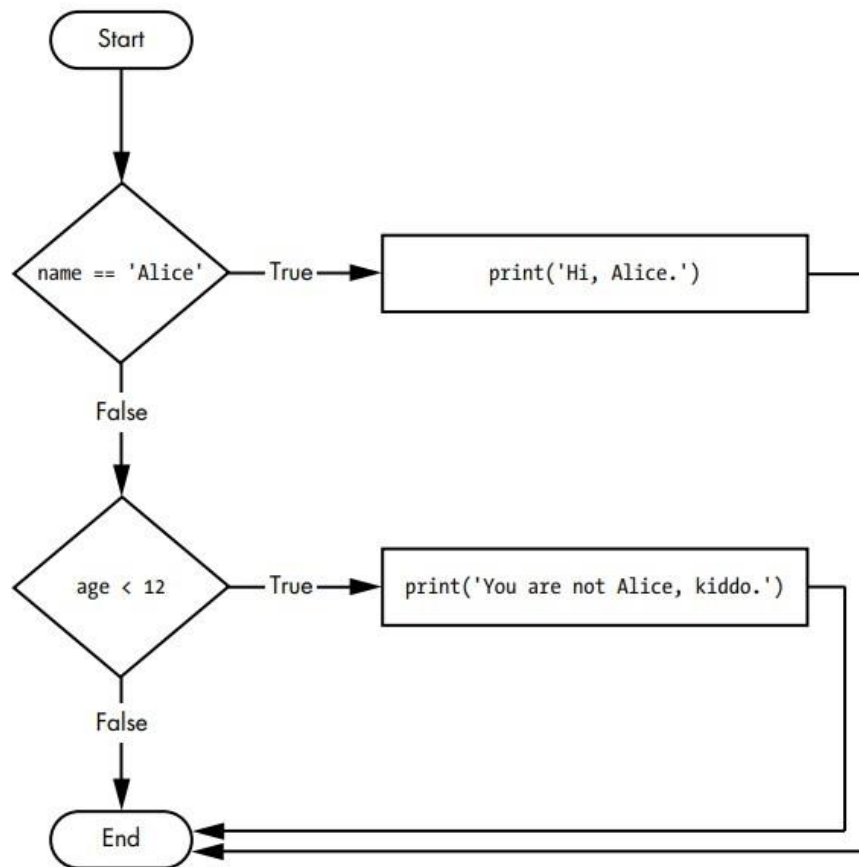


Figure 2-5: The flowchart for an `elif` statement

## if...elif...else statement

```
name = 'alice'
age = 10
if name == 'Alice' :
 print('Hi, Alice. ') # clause(block of code)
elif age<12:
 print('You are not Alice, Kiddo')
else:
 print('You are neither Alice nor a little kid. ')
```

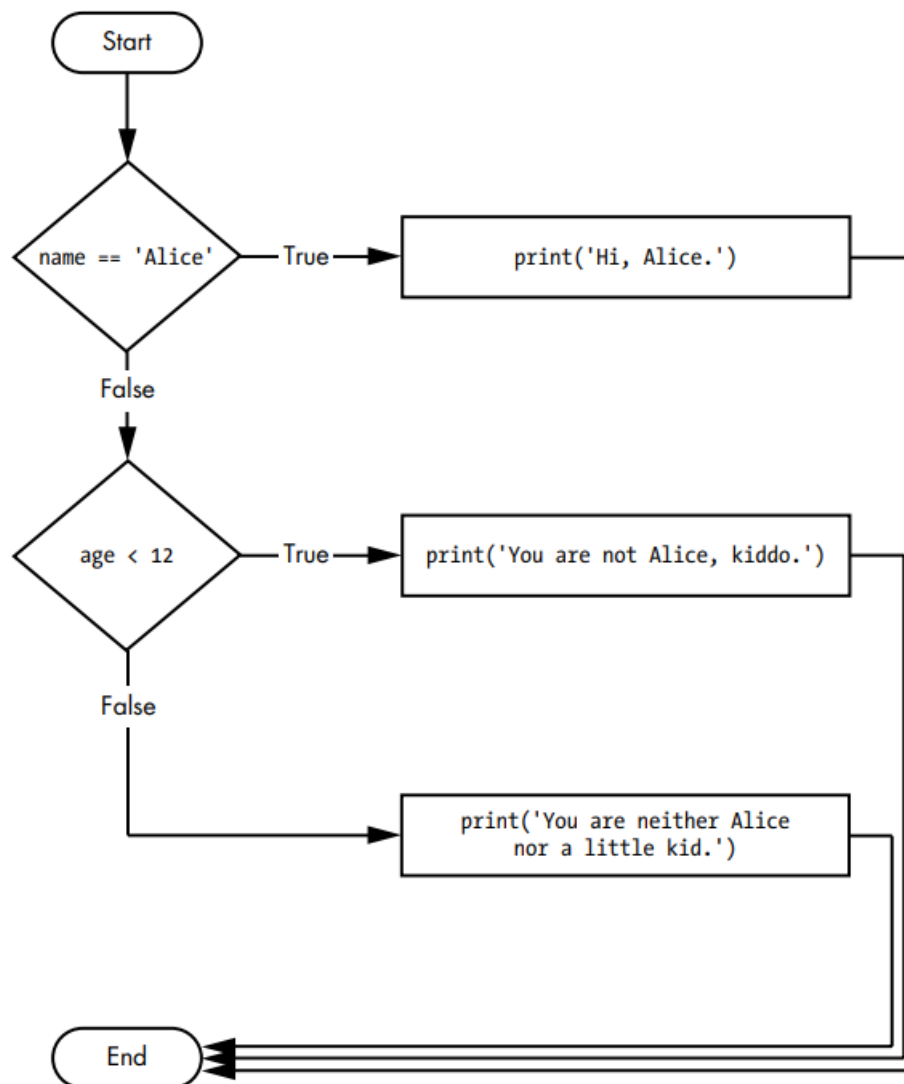


Figure shows the flowchart of if...elif...else

### while Loop Statements

- You can make a block of code execute over and over again with a while statement.
- The code in a while clause will be executed as long as the while statement's condition is True.

Syntax:

**while condition:**

**# body of while loop**

---

In code, a while statement always consists of the following:

- The **while** keyword
- A condition (that is, an expression that evaluates to True or False)
- A colon
- Starting on the next line, an **indented block of code** (called the while clause)

You can see that a while statement looks similar to an if statement. The difference is in how they behave.

- At the end of an if clause, the program execution continues after the if statement.
- But at the end of a while clause, the program execution jumps back to the start of the while statement.
- The while clause is often called the while loop or just the loop.

Let's look at an if statement and a while loop that use the same condition and take the same actions based on that condition.

Here is the code with an if statement:

---

```
spam = 0
if spam < 5:
 print('Hello, world.')
 spam = spam + 1
```

---

Here is the code with a while statement:

---

```
spam = 0
while spam < 5:
 print('Hello, world.')
 spam = spam + 1
```

---

- These statements are similar—both if and while check the value of spam, and if it's less than five, they print a message.
- But when you run these two code snippets, something very different happens for each one.
- For the if statement, the output is simply "Hello, world."
- But for the while statement, it's "Hello, world." repeated five times!

Take a look at the flowcharts for these two pieces of code, Figures 2-9 and 2-10, to see why this happens.

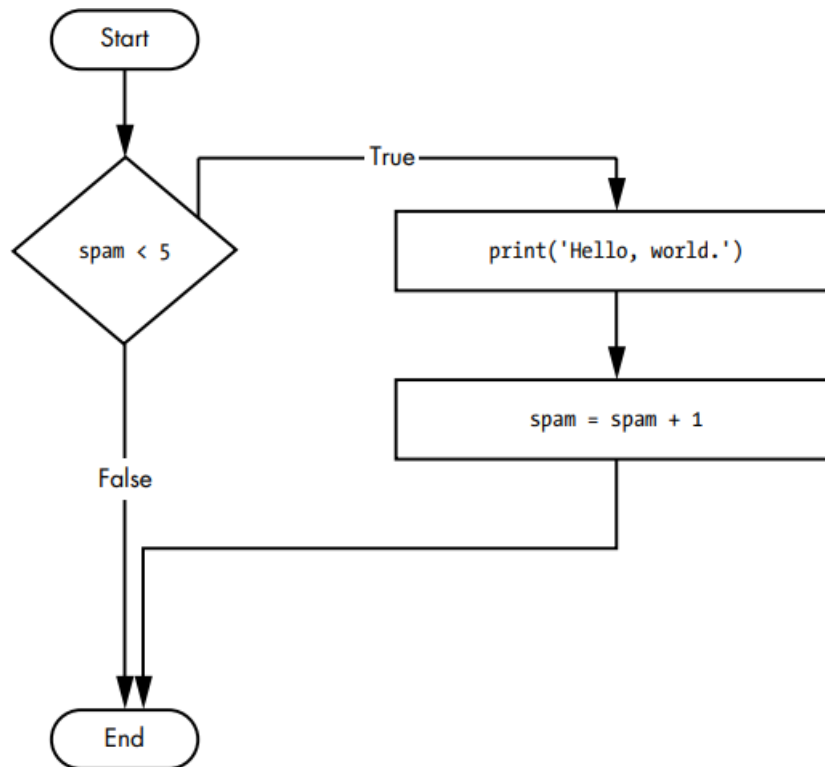


Figure 2-9: The flowchart for the *if* statement code

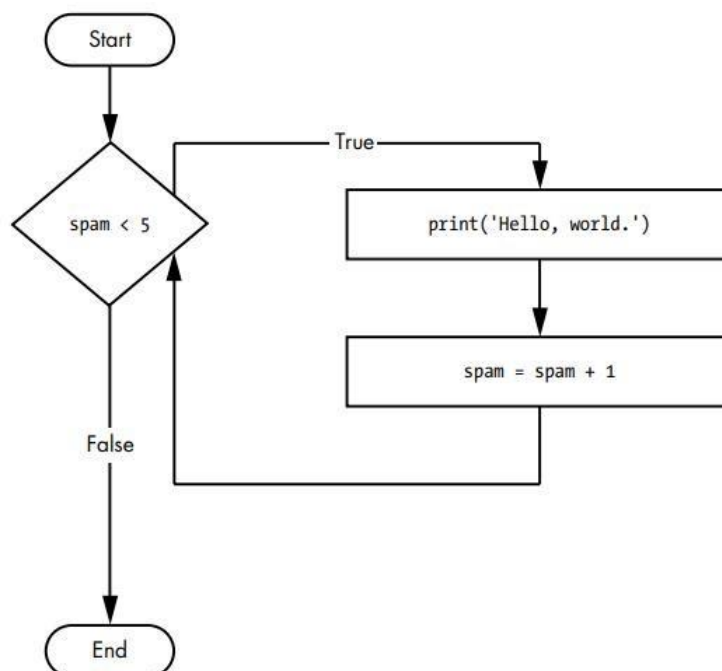


Figure 2-10: The flowchart for the *while* statement code

- 
- The code with the if statement checks the condition, and it prints Hello, world. only once if that condition is true.
  - The code with the while loop, on the other hand, will print it five times.
  - It stops after five prints because the integer in spam is incremented by one at the end of each loop iteration, which means that the loop will execute five times before `spam < 5` is False.
  - In the while loop, the condition is always checked at the start of each iteration (that is, each time the loop is executed).
  - If the condition is True, then the clause is executed, and afterward, the condition is checked again.
  - The first time the condition is found to be False, the while clause is skipped

---

## An Annoying while Loop

Here's a small example program that will keep asking you to type, literally, **your name**. Select **File ▶ New Window** to open a new file editor window, enter the following code, and save the file as *yourName.py*:

---

```
❶ name = ''
❷ while name != 'your name':
 print('Please type your name.')
❸ name = input()
❹ print('Thank you!')
```

---

First, the program sets the `name` variable ❶ to an empty string. This is so that the `name != 'your name'` condition will evaluate to `True` and the program execution will enter the `while` loop's clause ❷.

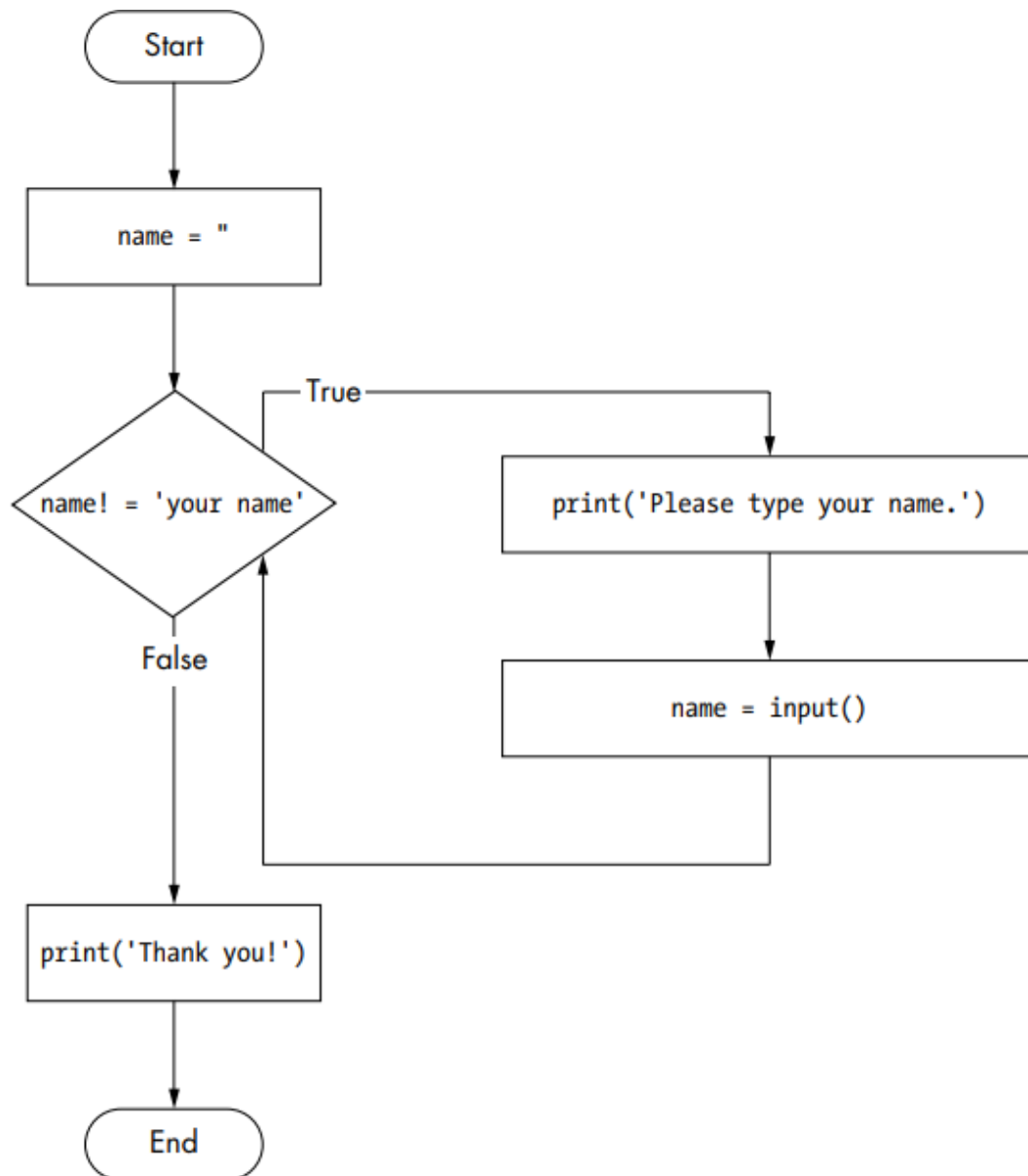
Flow Control **47**

---

The code inside this clause asks the user to type their name, which is assigned to the `name` variable ❸. Since this is the last line of the block, the execution moves back to the start of the `while` loop and reevaluates the condition. If the value in `name` is *not equal* to the string `'your name'`, then the condition is `True`, and the execution enters the `while` clause again.

But once the user types **your name**, the condition of the `while` loop will be `'your name' != 'your name'`, which evaluates to `False`. The condition is now `False`, and instead of the program execution reentering the `while` loop's clause, it skips past it and continues running the rest of the program ❹. Figure 2-11 shows a flowchart for the *yourName.py* program.

---



*Figure 2-11: A flowchart of the `yourName.py` program*

---

Now, let's see *yourName.py* in action. Press **F5** to run it, and enter something other than **your name** a few times before you give the program what it wants.

---

Please type your name.

**Al**

Please type your name.

**Albert**

---

Please type your name.

**%#@#%\*(^&!!!**

Please type your name.

**your name**

Thank you!

---

If you never enter **your name**, then the `while` loop's condition will never be `False`, and the program will just keep asking forever. Here, the `input()` call lets the user enter the right string to make the program move on. In other programs, the condition might never actually change, and that can be a problem. Let's look at how you can break out of a `while` loop.

## break Statements

- There is a shortcut to getting the program execution to break out of a `while` loop's clause early.
- If the execution reaches a `break` statement, it immediately exits the `while` loop's clause. In code, a `break` statement simply contains the `break` keyword.

---

```
❶ while True:
 print('Please type your name.')
❷ name = input()
❸ if name == 'your name':
❹ break
❺ print('Thank you!')
```

---

The first line ❶ creates an *infinite loop*; it is a while loop whose condition is always True. (The expression True, after all, always evaluates down to the value True.) The program execution will always enter the loop and will exit it only when a break statement is executed. (An infinite loop that *never* exits is a common programming bug.)

Just like before, this program asks the user to type **your name** ❷. Now, however, while the execution is still inside the while loop, an if statement gets executed ❸ to check whether name is equal to your name. If this condition is True, the break statement is run ❹, and the execution moves out of the loop to print('Thank you!') ❺. Otherwise, the if statement's clause with the break statement is skipped, which puts the execution at the end of the while loop. At this point, the program execution jumps back to the start of the while statement ❶ to recheck the condition. Since this condition is merely the True Boolean value, the execution enters the loop to ask the user to type **your name** again. See Figure 2-12 for the flowchart of this program.

Run *yourName2.py*, and enter the same text you entered for *yourName.py*. The rewritten program should respond in the same way as the original.

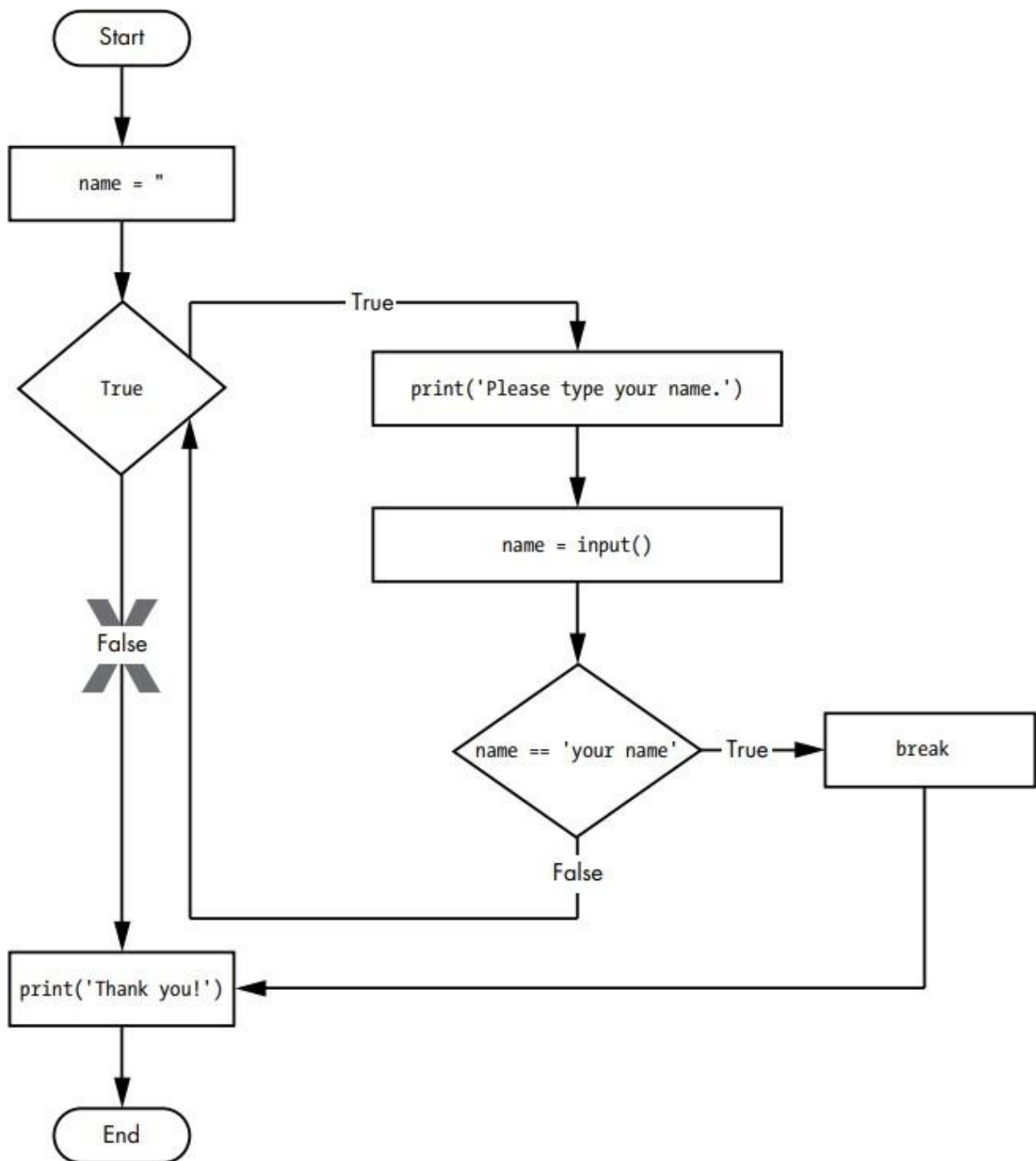


Figure 2-12: The flowchart for the `yourName2.py` program with an infinite loop. Note that the X path will logically never happen because the loop condition is always `True`.

---

## continue Statements

- Like break statements, continue statements are used inside loops.
- When the program execution reaches a continue statement, the program execution immediately jumps back to the start of the loop and re-evaluates the loop's condition. (This is also what happens when the execution reaches the end of the loop.)

---

```
while True:
 print('Who are you?')
 name = input()
 ❶ if name != 'Joe':
 ❷ continue
 print('Hello, Joe. What is the password? (It is a fish.)')
 ❸ password = input()
 if password == 'swordfish':
 ❹ break
 ❺ print('Access granted.')
```

---

If the user enters any name besides Joe ❶, the continue statement ❷ causes the program execution to jump back to the start of the loop. When it reevaluates the condition, the execution will always enter the loop, since the condition is simply the value True. Once they make it past that if statement, the user is asked for a password ❸. If the password entered is swordfish, then the break statement ❹ is run, and the execution jumps out of the while loop to print Access granted ❺. Otherwise, the execution continues to the end of the while loop, where it then jumps back to the start of the loop. See Figure 2-13 for this program's flowchart.

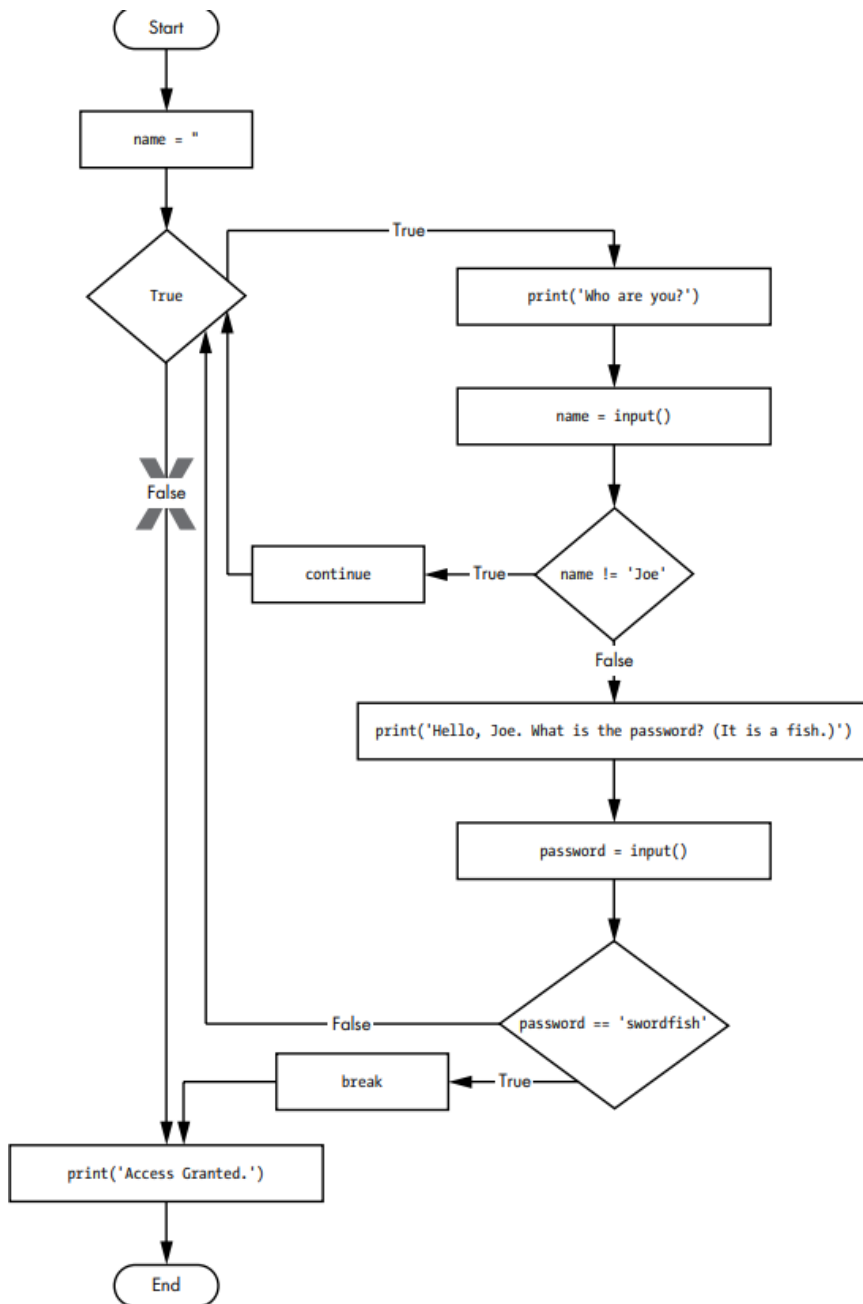


Figure 2-13: A flowchart for `swordfish.py`. The X path will logically never happen because the loop condition is always `True`.

---

## Trapped in an Infinite Loop?

If you ever run a program that has a bug causing it to get stuck in an infinite loop, press ctrl-C. This will send a Keyboard Interrupt error to your program and cause it to stop immediately

Ex.

```
while True:
 print('Hello world!')
```

- When you run this program, it will print Hello world! to the screen forever, because the while statement's condition is always True.
- In IDLE's interactive shell window, there are only two ways to stop this program: press ctrl-C or select Shell- Restart Shell from the menu.
- ctrl-C is handy if you ever want to terminate your program immediately, even if it's not stuck in an infinite loop

## for Loop

### for Loops with range() Function

The while loop keeps looping while its condition is True (which is the reason for its name), but what if you want to execute a block of code only a certain number of times? You can do this with a **for** loop statement and the **range()** function.

In code, a **for** statement looks something like **for i in range(5):** and always includes the following:

- The **for** keyword
- A variable name
- The **in** keyword
- A call to the **range()** method with up to three integers passed to it
- A colon
- Starting on the next line, an indented block of code (called the for clause)

```
print('My name is')
for i in range(5):
 print('Jimmy Five Times (' + str(i) + ')')
```

---

```
My name is
Jimmy Five Times (0)
Jimmy Five Times (1)
Jimmy Five Times (2)
Jimmy Five Times (3)
Jimmy Five Times (4)
```

---

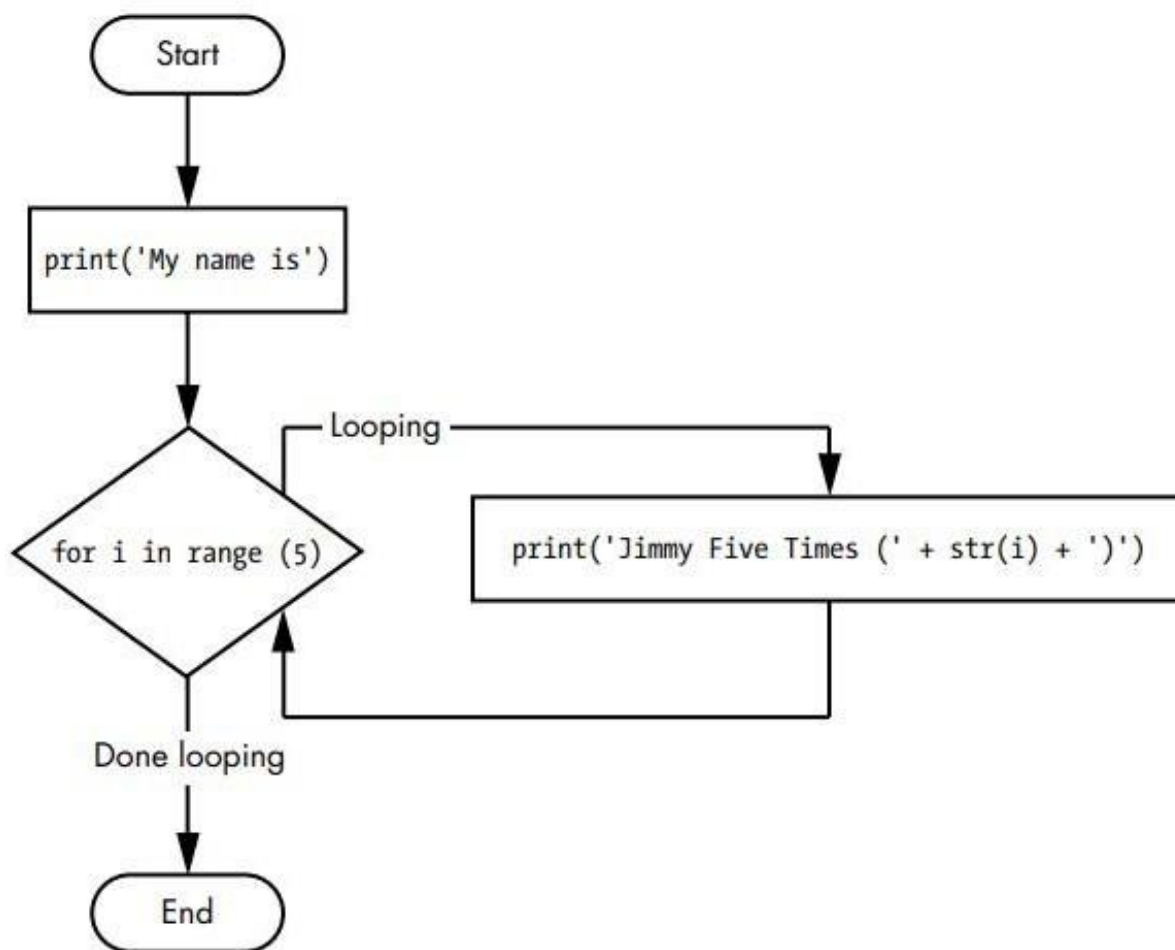


Figure 2-14: The flowchart for fiveTimes.py

### The Starting, Stopping, and Stepping Arguments to range()

---

➤ Some functions can be called with multiple arguments separated by a comma, and `range()` is one of them.

➤ This lets you change the integer passed to `range()` to follow any sequence of integers, including starting at a number other than zero.

**Ex:**

```
for i in range(12, 16,1):
 print(i)
```

**Output:**

```
12
13
14
15
```

The first argument will be where the for loop's variable starts, and the second argument will be up to, but not including, the number to stop at.

- The `range()` function can also be called with three arguments.
- The first two arguments will be the start and stop values, and the third will be the step argument.
- The step is the amount that the variable is increased by after each iteration.

```
for i in range(0, 10, 2):
 print(i)
```

So calling `range(0, 10, 2)` will count from zero to eight by intervals of two.

**Output:**

```
0
2
4
6
8
```

you can even use a negative number for the step argument to make the for loop count down instead of up.

```
for i in range(5, -1, -1):
 print(i)
```

---

Running a for loop to print i with range(5, -1, -1) should print from five down to zero.

**Output:**

5  
4  
3  
2  
1  
0

## 1.2.8 Importing Modules

- All Python programs can call a basic set of functions called built-in functions, including the print(), input(), and len() functions you've seen before.
- Python also comes with a set of modules called the standard library.
- Each module is a Python program that contains a related group of functions that can be embedded in your programs.
- For example, the math module has mathematics related functions, the random module has random number-related functions, and so on.

Before you can use the functions in a module, you must import the module with an import statement.

In code, an import statement consists of the following:

- The **import** keyword
- The name of the module
- Optionally, more module names, as long as they are separated by commas

Once you import a module, you can use all the cool functions of that module.

Let's give it a try with the random module, which will give us access to the random.randint() function.

```
import random
for i in range(5):
```

---

```
print(random.randint(1, 10))
```

Output:

```
4
1
8
4
1
```

- The `random.randint()` function call evaluates to a random integer value between the two integers that you pass it.
- Since `randint()` is in the `random` module, you must first type `random.` in front of the function name to tell Python to look for this function inside the `random` module.

Here's an example of an import statement that imports four different modules:

```
import random, sys, os, math
```

#### **from import Statements.**

- An alternative form of the import statement is composed of the `from` keyword, followed by the module name, the `import` keyword, and a star;
  - for example, **`from random import *`**.
  - With this form of import statement, calls to functions in `random` will not need the `random.` prefix.
  - However, using the full name makes for more readable code, so it is better to use the normal form of the import

### **1.2.9 Ending a Program Early with `sys.exit()`**

- The last flow control concept to cover is how to terminate the program.
- This always happens if the program execution reaches the bottom of the instructions.
- However, you can cause the program to terminate, or exit, by calling the `sys.exit()` function.
- Since this function is in the `sys` module, you have to **`import sys`** before your program can use it.

---

```
import sys

while True:
 print('Type exit to exit.')
 response = input()
 if response == 'exit':
 sys.exit()
 print('You typed ' + response + '.')
```

---

Run this program in IDLE. This program has an infinite loop with no break statement inside. The only way this program will end is if the user enters exit, causing `sys.exit()` to be called. When response is equal to exit, the program ends. Since the response variable is set by the `input()` function, the user must enter exit in order to stop the program.

## Practice Questions

1. What are the two values of the Boolean data type? How do you write them?
2. What are the three Boolean operators?
3. Write out the truth tables of each Boolean operator (that is, every possible combination of Boolean values for the operator and what they evaluate to).
4. What do the following expressions evaluate to?

---

```
(5 > 4) and (3 == 5)
not (5 > 4)
(5 > 4) or (3 == 5)
not ((5 > 4) or (3 == 5))
(True and True) and (True == False)
(not False) or (not True)
```

---

5. What are the six comparison operators?
6. What is the difference between the equal to operator and the assignment operator?
7. Explain what a condition is and where you would use one.

- 
8. Identify the three blocks in this code:
- 

```
spam = 0
if spam == 10:
 print('eggs')
 if spam > 5:
 print('bacon')
 else:
 print('ham')
 print('spam')
print('spam')
```

---

9. Write code that prints Hello if 1 is stored in spam, prints Howdy if 2 is stored in spam, and prints Greetings! if anything else is stored in spam.
10. What can you press if your program is stuck in an infinite loop?
11. What is the difference between break and continue?
12. What is the difference between range(10), range(0, 10), and range(0, 10, 1) in a for loop?
13. Write a short program that prints the numbers 1 to 10 using a for loop. Then write an equivalent program that prints the numbers 1 to 10 using a while loop.
14. If you had a function named bacon() inside a module named spam, how would you call it after importing spam?

---

## Module-1

### 1.3 Functions:

Python provides several built-in functions like `input()`, `print()`, `len()`.

Apart from that python allows to write your own functions.

A function is like a mini-program within a program.

To better understand how functions work, let's take one example.

---

```
❶ def hello():
❷ print('Howdy!')
 print('Howdy!!!')
 print('Hello there.')

❸ hello()
 hello()
 hello()
```

---

- The first line is a `def` statement (1), which defines a function named `hello()`.
- The code in the block that follows the `def` statement (2) is the body of the function.

- 
- This code is executed when the function is called, not when the function is first defined.
  - The `hello()` lines after the function (3) are function calls.
  - In code, a function call is just the function's name followed by parentheses, possibly with some number of arguments in between the parentheses.
  - When the program execution reaches these calls, it will jump to the top line in the function and begin executing the code there.
  - When it reaches the end of the function, the execution returns to the line that called the function and continues moving through the code as before.
  - Since this program calls `hello()` three times, the code in the `hello()` function is executed three times.
  - When you run this program, the output looks like this:

---

```
Howdy!
Howdy!!!
Hello there.
Howdy!
Howdy!!!
Hello there.
Howdy!
Howdy!!!
Hello there.
```

---

A major purpose of functions is to group code that gets executed multiple times. Without a function defined, you would have to copy and paste this code each time, and the program would look like this:

---

```
print('Howdy!')
print('Howdy!!!')
print('Hello there.')
print('Howdy!')
print('Howdy!!!')
print('Hello there.')
print('Howdy!')
print('Howdy!!!')
print('Hello there.')
```

---

In general, you always want to avoid duplicating code, because if you ever decide to update the code—if, for example, you find a bug you need to fix—you'll have to remember to change the code everywhere you copied it.

As you get more programming experience, you'll often find yourself *deduplicating* code, which means getting rid of duplicated or copy-and-pasted code. Deduplication makes your programs shorter, easier to read, and easier to update.

### 1.3.1 def Statements with Parameter

- When you call the `print()` or `len()` function, you pass in values, called arguments in this context, by typing them between the parentheses.
- You can also define your own functions that accept arguments.

For example:

---

```
❶ def hello(name):
❷ print('Hello ' + name)

❸ hello('Alice')
 hello('Bob')
```

---

When you run this program, the output looks like this:

---

```
Hello Alice
Hello Bob
```

---

- The definition of the hello() function in this program has a parameter called name (1).
- A parameter is a variable that an argument is stored in when a function is called.
- The first time the hello() function is called, it's with the argument 'Alice' (3).
- The program execution enters the function, and the variable name is automatically set to 'Alice', which is what gets printed by the print() statement (2).

One special thing to note about parameters is that the value stored in a parameter is forgotten when the function returns.

For example, if you added print(name) after hello('Bob') in the previous program, the program would give you a NameError because there is no variable named name. This variable was destroyed after the function call hello('Bob') had returned, so print(name) would refer to a name variable that does not exist.

### 1.3.2 Return Values and return Statements

- When you call the len() function and pass it an argument such as 'Hello', the function call evaluates to the integer value 5, which is the length of the string you passed it.
- In general, the value that a function call evaluates to is called the return value of the function.
- When creating a function using the def statement, you can specify what the return value should be with a return statement.

---

A return statement consists of the following:

- The **return** keyword
  - The value or expression that the function should return
- When an expression is used with a return statement, the return value is what this expression evaluates to.
  - For example, the following program defines a function that returns a different string depending on what number it is passed as an argument.

---

```
❶ import random

❷ def getAnswer(answerNumber):
❸ if answerNumber == 1:
 return 'It is certain'
 elif answerNumber == 2:
 return 'It is decidedly so'
 elif answerNumber == 3:
 return 'Yes'
 elif answerNumber == 4:
 return 'Reply hazy try again'
 elif answerNumber == 5:
 return 'Ask again later'
 elif answerNumber == 6:
 return 'Concentrate and ask again'
 elif answerNumber == 7:
 return 'My reply is no'
 elif answerNumber == 8:
 return 'Outlook not so good'
 elif answerNumber == 9:
 return 'Very doubtful'

❹ r = random.randint(1, 9)
❺ fortune = getAnswer(r)
❻ print(fortune)
```

---

- When this program starts, Python first imports the random module (1).
- Then the getAnswer() function is defined (2).

- 
- Because the function is being defined (and not called), the execution skips over the code in it.
  - Next, the `random.randint()` function is called with two arguments, 1 and 9 (4).
  - It evaluates to a random integer between 1 and 9 (including 1 and 9 themselves), and this value is stored in a variable named `r`.
  - The `getAnswer()` function is called with `r` as the argument (5).
  - The program execution moves to the top of the `getAnswer()` function (3), and the value `r` is stored in a parameter named `answerNumber`.
  - Then, depending on this value in `answerNumber`, the function returns one of many possible string values.
  - The program execution returns to the line at the bottom of the program that originally called `getAnswer()` (5).
  - The returned string is assigned to a variable named `fortune`, which then gets passed to a `print()` call (6) and is printed to the screen.

### 1.3.3 The None Value

- In Python there is a value called `None`, which represents the absence of a value.
- `None` is the only value of the `NoneType` data type. (Other programming languages might call this value `null`, `nil`, or `undefined`.)
- Just like the Boolean `True` and `False` values, `None` must be typed with a capital `N`.
- This value-without-a-value can be helpful when you need to store something that won't be confused for a real value in a variable.
- One place where `None` is used is as the return value of `print()`.
- The `print()` function displays text on the screen, but it doesn't need to return anything in the same way `len()` or `input()` does.
- But since all function calls need to evaluate to a return value, `print()` returns `None`.

Example:

---

```
>>> spam = print('Hello!')
Hello!
>>> None == spam
True
```

---

## Keyword Arguments and print()

- Most arguments are identified by their position in the function call.
- For example, `random.randint(1, 10)` is different from `random.randint(10, 1)`.
- The function call `random.randint(1, 10)` will return a random integer between 1 and 10, because the first argument is the low end of the range and the second argument is the high end (while `random.randint(10, 1)` causes an error).

However, keyword arguments are identified by the keyword put before them in the function call.

Keyword arguments are often used for optional parameters.

For example, the `print()` function has the optional parameters `end` and `sep` to specify what should be printed at the end of its arguments and between its arguments (separating them), respectively.

If you ran the following program

---

```
print('Hello')
print('World')
```

---

the output would look like this:

---

```
Hello
World
```

---

- 
- The two strings appear on separate lines because the `print()` function automatically adds a newline character to the end of the string it is passed.
  - However, you can set the `end` keyword argument to change this to a different string. For example, if the program were this:

---

```
print('Hello', end='')
print('World')
```

---

the output would look like this:

---

```
HelloWorld
```

---

Similarly, when you pass multiple string values to `print()`, the function will automatically separate them with a single space.

Enter the following into the interactive shell:

---

```
>>> print('cats', 'dogs', 'mice')
cats dogs mice
```

---

But you could replace the default separating string by passing the `sep` keyword argument. Enter the following into the interactive shell:

---

```
>>> print('cats', 'dogs', 'mice', sep=',')
cats,dogs,mice
```

---

## Local and Global Scope

- Parameters and variables that are assigned in a called function are said to exist in that function's local scope.
- Variables that are assigned outside all functions are said to exist in the global scope.
- A variable that exists in a local scope is called a local variable, while a variable that exists in the global scope is called a global variable.
- A variable must be one or the other; it cannot be both local and global.
- Think of a scope as a container for variables.

- 
- When a scope is destroyed, all the values stored in the scope's variables are forgotten.
  - There is only one global scope, and it is created when your program begins. When your program terminates, the global scope is destroyed, and all its variables are forgotten.
  - Otherwise, the next time you ran your program, the variables would remember their values from the last time you ran it.
- 
- A local scope is created whenever a function is called.
  - Any variables assigned in this function exist within the local scope.
  - When the function returns, the local scope is destroyed, and these variables are forgotten.
  - The next time you call this function, the local variables will not remember the values stored in them from the last time the function was called.

Scopes matter for several reasons:

- Code in the global scope cannot use any local variables.
- However, a local scope can access global variables.
- Code in a function's local scope cannot use variables in any other local scope.
- You can use the same name for different variables if they are in different scopes. That is, there can be a local variable named spam and a global variable also named spam.

### Local Variables Cannot Be Used in the Global Scope

```
def spam():
 eggs = 31337
spam()
print(eggs)
```

---

If you run this program, the output will look like this:

---

```
Traceback (most recent call last):
 File "C:/test3784.py", line 4, in <module>
 print(eggs)
NameError: name 'eggs' is not defined
```

---

The error happens because the `eggs` variable exists only in the local scope created when `spam()` is called. Once the program execution returns from `spam`, that local scope is destroyed, and there is no longer a variable named `eggs`. So when your program tries to run `print(eggs)`, Python gives you an error saying that `eggs` is not defined. This makes sense if you think about it; when the program execution is in the global scope, no local scopes exist, so there can't be any local variables. This is why only global variables can be used in the global scope.

### **Local Scopes Cannot Use Variables in Other Local Scopes**

A new local scope is created whenever a function is called, including when a function is called from another function. Consider this program:

---

## ***Local Scopes Cannot Use Variables in Other Local Scopes***

A new local scope is created whenever a function is called, including when a function is called from another function. Consider this program:

---

```
def spam():
 ❶ eggs = 99
 ❷ bacon()
 ❸ print(eggs)

def bacon():
 ham = 101
 ❹ eggs = 0

❺ spam()
```

---

When the program starts, the `spam()` function is called ❺, and a local scope is created. The local variable `eggs` ❶ is set to 99. Then the `bacon()` function is called ❷, and a second local scope is created. Multiple local scopes can exist at the same time. In this new local scope, the local variable `ham` is set to 101, and a local variable `eggs`—which is different from the one in `spam()`'s local scope—is also created ❹ and set to 0.

When `bacon()` returns, the local scope for that call is destroyed. The program execution continues in the `spam()` function to print the value of `eggs` ❸, and since the local scope for the call to `spam()` still exists here, the `eggs` variable is set to 99. This is what the program prints.

The upshot is that local variables in one function are completely separate from the local variables in another function.

## ***Global Variables Can Be Read from a Local Scope***

Consider the following program:

---

```
def spam():
 print(eggs)
eggs = 42
spam()
print(eggs)
```

---

Since there is no parameter named `eggs` or any code that assigns `eggs` a value in the `spam()` function, when `eggs` is used in `spam()`, Python considers it a reference to the global variable `eggs`. This is why 42 is printed when the previous program is run.

---

## Local and Global Variables with the Same Name

To simplify your life, avoid using local variables that have the same name as a global variable or another local variable. But technically, it's perfectly legal to do so in Python. To see what happens, type the following code into the file editor and save it as *sameName.py*:

---

```
def spam():
❶ eggs = 'spam local'
 print(eggs) # prints 'spam local'

def bacon():
❷ eggs = 'bacon local'
 print(eggs) # prints 'bacon local'
 spam()
 print(eggs) # prints 'bacon local'

❸ eggs = 'global'
 bacon()
 print(eggs) # prints 'global'
```

---

When you run this program, it outputs the following:

---

```
bacon local
spam local
bacon local
global
```

---

There are actually three different variables in this program, but confusingly they are all named `eggs`. The variables are as follows:

- ❶ A variable named `eggs` that exists in a local scope when `spam()` is called.
- ❷ A variable named `eggs` that exists in a local scope when `bacon()` is called.
- ❸ A variable named `eggs` that exists in the global scope.

Since these three separate variables all have the same name, it can be confusing to keep track of which one is being used at any given time.

This is why you should avoid using the same variable name in different scopes.

---

## The global Statement

If you need to modify a global variable from within a function, use the `global` statement. If you have a line such as `global eggs` at the top of a function, it tells Python, “In this function, `eggs` refers to the global variable, so don’t create a local variable with this name.” For example, type the following code into the file editor and save it as *sameName2.py*:

---

```
def spam():
 ❶ global eggs
 ❷ eggs = 'spam'

eggs = 'global'
spam()
print(eggs)
```

---

When you run this program, the final `print()` call will output this:

---

```
spam
```

---

Because `eggs` is declared `global` at the top of `spam()` ❶, when `eggs` is set to `'spam'` ❷, this assignment is done to the globally scoped `spam`. No local `spam` variable is created.

There are four rules to tell whether a variable is in a local scope or global scope:

1. If a variable is being used in the global scope (that is, outside of all functions), then it is always a global variable.
2. If there is a global statement for that variable in a function, it is a global variable.
3. Otherwise, if the variable is used in an assignment statement in the function, it is a local variable.
4. But if the variable is not used in an assignment statement, it is a global variable.

---

```
def spam():
❶ global eggs
 eggs = 'spam' # this is the global

def bacon():
❷ eggs = 'bacon' # this is a local
```

---

```
def ham():
❸ print(eggs) # this is the global

eggs = 42 # this is the global
spam()
print(eggs)
```

---

In the `spam()` function, `eggs` is the global `eggs` variable, because there's a global statement for `eggs` at the beginning of the function ❶. In `bacon()`, `eggs` is a local variable, because there's an assignment statement for it in that function ❷. In `ham()` ❸, `eggs` is the global variable, because there is no assignment statement or global statement for it in that function. If you run *sameName3.py*, the output will look like this:

---

```
spam
```

---

In a function, a variable will either always be global or always be local. There's no way that the code in a function can use a local variable named `eggs` and then later in that same function use the global `eggs` variable.

## Exception Handling

- Right now, getting an error, or exception, in your Python program means the entire program will crash.
- You don't want this to happen in real-world programs.
- Instead, you want the program to detect errors, handle them, and then continue to run.

---

For example, consider the following program, which has a “divide by zero” error.

---

```
def spam(divideBy):
 return 42 / divideBy

print(spam(2))
print(spam(12))
print(spam(0))
print(spam(1))
```

---

We’ve defined a function called `spam`, given it a parameter, and then printed the value of that function with various parameters to see what happens.

This is the output you get when you run the previous code:

---

```
21.0
3.5
Traceback (most recent call last):
 File "C:/zeroDivide.py", line 6, in <module>
 print(spam(0))
 File "C:/zeroDivide.py", line 2, in spam
 return 42 / divideBy
ZeroDivisionError: division by zero
```

---

A `ZeroDivisionError` happens whenever you try to divide a number by zero.

From the line number given in the error message, you know that the return statement in `spam()` is causing an error.

---

Errors can be handled with try and except statements. The code that could potentially have an error is put in a try clause. The program execution moves to the start of a following except clause if an error happens.

You can put the previous divide-by-zero code in a try clause and have an except clause contain code to handle what happens when this error occurs.

---

```
def spam(divideBy):
 try:
 return 42 / divideBy
 except ZeroDivisionError:
 print('Error: Invalid argument.')

print(spam(2))
print(spam(12))
print(spam(0))
print(spam(1))
```

---

When code in a try clause causes an error, the program execution immediately moves to the code in the except clause. After running that code, the execution continues as normal. The output of the previous program is as follows:

---

```
21.0
3.5
Error: Invalid argument.
None
42.0
```

---

---

Note that any errors that occur in function calls in a try block will also be caught. Consider the following program, which instead has the `spam()` calls in the try block:

---

```
def spam(divideBy):
 return 42 / divideBy

try:
 print(spam(2))
 print(spam(12))
 print(spam(0))
 print(spam(1))
except ZeroDivisionError:
 print('Error: Invalid argument.')
```

---

When this program is run, the output looks like this:

---

```
21.0
3.5
Error: Invalid argument.
```

---

---

The reason `print(spam(1))` is never executed is because once the execution jumps to the code in the except clause, it does not return to the try clause. Instead, it just continues moving down as normal.

---

## A Short Program: Guess the number

The toy examples I've show you so far are useful for introducing basic concepts, but now let's see how everything you've learned comes together in a more complete program. In this section, I'll show you a simple "guess the number" game. When you run this program, the output will look something like this:

---

```
I am thinking of a number between 1 and 20.
Take a guess.
10
Your guess is too low.
Take a guess.
15
Your guess is too low.
Take a guess.
17
Your guess is too high.
Take a guess.
16
Good job! You guessed my number in 4 guesses!
```

---

Type the following code

---

```
This is a guess the number game.
import random
secretNumber = random.randint(1, 20)
print('I am thinking of a number between 1 and 20.')

Ask the player to guess 6 times.
for guessesTaken in range(1, 7):
 print('Take a guess.')
 guess = int(input())

 if guess < secretNumber:
 print('Your guess is too low.')
 elif guess > secretNumber:
 print('Your guess is too high.')
 else:
 break # This condition is the correct guess!

if guess == secretNumber:
 print('Good job! You guessed my number in ' + str(guessesTaken) + ' guesses!')
else:
 print('Nope. The number I was thinking of was ' + str(secretNumber))
```

---

---

Let's look at this code line by line, starting at the top.

---

```
This is a guess the number game.
import random
secretNumber = random.randint(1, 20)
```

---

First, a comment at the top of the code explains what the program does. Then, the program imports the `random` module so that it can use the `random.randint()` function to generate a number for the user to guess. The return value, a random integer between 1 and 20, is stored in the variable `secretNumber`.

---

```
print('I am thinking of a number between 1 and 20.')

Ask the player to guess 6 times.
for guessesTaken in range(1, 7):
 print('Take a guess.')
 guess = int(input())
```

---

The program tells the player that it has come up with a secret number and will give the player six chances to guess it. The code that lets the player enter a guess and checks that guess is in a `for` loop that will loop at most six times. The first thing that happens in the loop is that the player types in a guess. Since `input()` returns a string, its return value is passed straight into `int()`, which translates the string into an integer value. This gets stored in a variable named `guess`.

---

```
 if guess < secretNumber:
 print('Your guess is too low.')
 elif guess > secretNumber:
 print('Your guess is too high.')
```

---

These few lines of code check to see whether the guess is less than or greater than the secret number. In either case, a hint is printed to the screen.

---

```
 else:
 break # This condition is the correct guess!
```

---

If the guess is neither higher nor lower than the secret number, then it must be equal to the secret number, in which case you want the program execution to break out of the `for` loop.

---

---

```
if guess == secretNumber:
 print('Good job! You guessed my number in ' + str(guessesTaken) + ' guesses!')
else:
 print('Nope. The number I was thinking of was ' + str(secretNumber))
```

---

After the for loop, the previous if...else statement checks whether the player has correctly guessed the number and prints an appropriate message to the screen. In both cases, the program displays a variable that contains

Functions **75**

an integer value (`guessesTaken` and `secretNumber`). Since it must concatenate these integer values to strings, it passes these variables to the `str()` function, which returns the string value form of these integers. Now these strings can be concatenated with the `+` operators before finally being passed to the `print()` function call.

## Practice Questions

1. Why are functions advantageous to have in your programs?
2. When does the code in a function execute: when the function is defined or when the function is called?
3. What statement creates a function?
4. What is the difference between a function and a function call?
5. How many global scopes are there in a Python program? How many local scopes?
6. What happens to variables in a local scope when the function call returns?
7. What is a return value? Can a return value be part of an expression?
8. If a function does not have a return statement, what is the return value of a call to that function?
9. How can you force a variable in a function to refer to the global variable?
10. What is the data type of `None`?
11. What does the `import areallyourpetsnamederic` statement do?
12. If you had a function named `bacon()` in a module named `spam`, how would you call it after importing `spam`?
13. How can you prevent a program from crashing when it gets an error?
14. What goes in the `try` clause? What goes in the `except` clause?

